

Работа с камерой ESP32-CAM

В прошлом уроке мы смогли отправить данные и получить ответ с сервера, запущенного с вашего компьютера. Теперь займемся тем, что будем получать изображение с камеры и сохранять его с помощью python-скрипта.

1. Рефакторинг кода

Первым делом займемся тем, что сделаем код более удобным для взаимодействия. Сегодня мы будем добавлять достаточно много кода, который сильно увеличит листинг текущего проекта. Поэтому разделим куски кода по файлам. Откройте код предыдущего занятия (`serverConnection.ino`).

Первое, что мы сделаем, перенесем в отдельный файл код, связанный с Wi-Fi подключением - функции `requestDataFromServer()`, `sendDataToServer()`, `connectToWifi()`.

Для удобства откройте **VSCode** в папке с проектом и создайте файл, например, `WifiManager.hpp`, в котором мы создадим класс **WifiManager**, который будет описывать методы взаимодействия с Wi-Fi:

```
#ifndef WIFIMANAGER_HPP_
#define WIFIMANAGER_HPP_

#include <WiFi.h>           // переносим библиотеки
#include <HTTPClient.h>
#include "esp_camera.h"

#include "System.hpp"

// Описание класса
class WifiManager {
public:
    WifiManager() // Конструктор класса
    {}
};

#endif /// WIFIMANAGER_HPP_
```

Конструктор класса - это функция, которая вызывается при создании объекта класса. Он имеет тоже название, что и сам класс. Перенесите функции, названные выше внутрь класса, чтобы мы могли с ними взаимодействовать только через класс **WifiManager** (функцию *sendDataToServer()* переименуйте, как показано ниже):

```
class WifiManager {
public:
    WifiManager() // Конструктор класса
    {}

    // Функция подключения к Wi-Fi
    void connectToWiFi() // Публичный метод
    {
        // Перенесите код
    }

private: // модификатор видимости составляющих класса
// Прием всех данных с сервера
    void requestDataFromServer() // Непубличный метод
    {
        // Перенесите код
    }
// в прошлом sendDataToServer - отправка данных об ESP
    void sendEspData() // Непубличный метод
    {
        // Перенесите код
    }
};
```

Как вы можете увидеть, у нас появилось неизвестные нам слова “private” и “public” - это модификаторы видимости составляющих класса извне. Если мы хотим взаимодействовать с полями и методами класса извне, то мы должны их определить ниже “public”, если не хотим, то “private”. При использовании “private” все взаимодействие может происходить только внутри класса.

```
WifiManager wifi; // Создание объекта типа WifiManager
wifi.connectToWiFi(); // Все OK
wifi.sendEspData(); // Ошибка - метод не доступен из этой области видимости
```

Перенесите объявление *ssid*, *password* и *serverIP* внутрь класса, а метод `connectToWifi()` в приватную область:

```
class WifiManager {  
private:  
    // Настройки Wi-Fi сети  
    // const char* ssid = "ИМЯ_ВАШЕЙ_СЕТИ";    // Название Wi-Fi сети  
    // const char* password = "ПАРОЛЬ_СЕТИ";    // Пароль от Wi-Fi  
  
    // Адрес сервера  
    // const char* serverIP = "192.168.14.215"; // Адрес вашего сервера
```

Так как у нас все методы не доступны на данный момент для взаимодействия, мы напишем функции, которые позволят нам с ними взаимодействовать - их нужно **добавить в публичную область видимости**.

```
public:  
    void request()  
    {  
        Serial.println("\nЗАПРАШИВАЮ ДАННЫЕ С СЕРВЕРА...");  
        requestDataFromServer();  
    }  
  
    void send()  
    {  
        Serial.println("ОТПРАВКА ДАННЫХ НА СЕРВЕР");  
        sendEspData();  
    }  
  
    void connect()  
    {  
        connectToWifi();  
    }
```

У нас будет два метода отправки данных на сервер: данные об ESP32 и данные с камеры. Зададим перечисления сразу после `serverIP`:

```
public:  
    enum class SendFormat {  
        ESP_DATA,  
        CAMERA_DATA  
    };
```

С помощью перечисления мы будем указывать, что мы хотим отправить на данный момент:

```
void send(SendFormat aFormat)
{
    Serial.println("ОТПРАВКА ДАННЫХ НА СЕРВЕР");
    switch (aFormat)
    {
        case SendFormat::ESP_DATA:
            sendEspData();
            break;

        case SendFormat::CAMERA_DATA:
            // Здесь будет вызов метода отправки изображения - sendCapture()
            break;

        default:
            break;
    }
}
```

Теперь реализуем функцию отправки изображения на сервер:

```
void sendCapture(camera_fb_t* aFb)
{
    Serial.println("ОТПРАВКА ИЗОБРАЖЕНИЯ");
    if (WiFi.status() != WL_CONNECTED) {
        Serial.println("Ошибка: нет подключения к WiFi!");
        return;
    }

    HTTPClient http;

    String url = "http://" + String(serverIP) + ":8080";
    http.begin(url);
    http.setTimeout(5000); // Таймаут 5 секунд

    // Подготовка multipart/form-data
    String boundary = "ESP32CAM-" + String(millis());
    String contentType = "multipart/form-data; boundary=" + boundary;
    http.addHeader("Content-Type", contentType);
```

```

// Формирование тела запроса
String bodyStart = "--" + boundary + "\r\n";
bodyStart += "Content-Disposition: form-data; name=\"imageFile\"";
filename=\"frame_\" +
    String(frameCount) + ".jpg\" \r\n";
bodyStart += "Content-Type: image/jpeg\r\n\r\n";

String bodyEnd = "\r\n--" + boundary + "--\r\n";

// Сборка полного пакета
size_t totalSize = bodyStart.length() + aFb->len + bodyEnd.length();
uint8_t* payload = (uint8_t*)malloc(totalSize);

if (!payload) {
    Serial.println("Ошибка выделения памяти для отправки");
    http.end();
    return;
}

memcpy(payload, bodyStart.c_str(), bodyStart.length());
memcpy(payload + bodyStart.length(), aFb->buf, aFb->len);
memcpy(payload + bodyStart.length() + aFb->len, bodyEnd.c_str(), bodyEnd.length());

// Отправка
Serial.printf("Отправка кадра #%d (%d байт)... ", frameCount, aFb->len);
int httpCode = http.POST(payload, totalSize);

free(payload);

if (httpCode == 200) {
    String response = http.getString();
    Serial.printf("HTTP %d - %s\n", httpCode, response.c_str());
    http.end();
    return;
} else {
    Serial.printf("HTTP %d\n", httpCode);
    http.end();
    return;
}
}

```

Функция работает по тому же принципу, как и *sendEspData()*, отличие у них в том, что пакеты данных, которые они формируют - отличаются, а так

же текущая функция принимает в качестве аргумента переменную типа *camera_fb_t*, которая хранит данные о полученном изображении с камеры. Останавливаться на этом отдельно не будем - **просто скопируйте код**. Добавим нашу функцию внутрь функции *send()*:

```
void send(SendFormat aFormat, camera_fb_t* aFb = nullptr)
{
    Serial.println("ОТПРАВКА ДАННЫХ НА СЕРВЕР");
    switch (aFormat)
    {
        case SendFormat::ESP_DATA:
            sendEspData();
            break;

        case SendFormat::CAMERA_DATA:
            sendCapture(aFb);
            break;

        default:
            break;
    }
}
```

Теперь у функции *send()*, появился параметр, который не обязательно задавать, т.к. по умолчанию он будет принимать значение “нулевого указателя” - *camera_fb_t* aFb = nullptr*.

Код по взаимодействию с Wi-Fi перенесем в функцию *process()*:

```
void process(SendFormat aFormat, camera_fb_t* aFb)
{
    if (WiFi.status() == WL_CONNECTED) {
        Serial.println("Все еще подключены к Wi-Fi");
        send(aFormat, aFb);

        request();
    } else {
        Serial.println("Потеряно соединение с Wi-Fi");
        connectToWiFi();
    }
}
```

Перенесите подключаемые файлы в текущий файл:

```
#include <WiFi.h>
#include <HTTPClient.h>
#include "esp_camera.h"
```

Пока с Wi-Fi закончим, но мы к нему вернемся. У нас еще осталась функция *blinkLed()* - мы ее тоже вынесем в отдельный файл - **System.hpp**:

```
#ifndef SYSTEM_HPP_
#define SYSTEM_HPP_

// Пины для светодиода (для индикации)
#define LED_PIN 4 // Встроенный светодиод на ESP32-CAM

void blinkLED(int count, int delayTime)
{
    for (int i = 0; i < count * 2; i++) {
        digitalWrite(LED_PIN, !digitalRead(LED_PIN));
        delay(delayTime);
    }
}

#endif /// SYSTEM_HPP_
```

Теперь можете из основного файла удалить все, что мы перенесли в другие файлы, после чего подключите файлы, которые мы создали:

```
// Подключаем необходимые библиотеки
#include "WifiManager.hpp"
```

System.hpp подключите в файл **WifiManager.hpp**.

2. Камера

После того, как мы “причесали” проект, можем заняться камерой. Создайте файл **Camera.hpp** в нем мы будем реализовывать методы взаимодействия с камерой. Подключите необходимые библиотеки:

```
#ifndef CAMSETTINGS_HPP_
#define CAMSETTINGS_HPP_
```

```
// перед `#include "esp_camera.h"
#define CAMERA_MODEL_AI_THINKER

#include "soc/soc.h"          // Для отключения детектора понижения питания
#include "soc/rtc_cntl_reg.h" // Для отключения детектора понижения питания
#include "esp_camera.h"
```

Создадим класс **Camera**, в который мы сразу добавим конструктор класса и данные о пинах, подключенных к камере:

```
class Camera {
private:
    // Конфигурация пинов для AI Thinker ESP32-CAM
    #define PWDN_GPIO_NUM  32
    #define RESET_GPIO_NUM -1
    #define XCLK_GPIO_NUM  0
    #define SIOD_GPIO_NUM  26
    #define SIOC_GPIO_NUM  27
    #define Y9_GPIO_NUM    35
    #define Y8_GPIO_NUM    34
    #define Y7_GPIO_NUM    39
    #define Y6_GPIO_NUM    36
    #define Y5_GPIO_NUM    21
    #define Y4_GPIO_NUM    19
    #define Y3_GPIO_NUM    18
    #define Y2_GPIO_NUM     5
    #define VSYNC_GPIO_NUM 25
    #define HREF_GPIO_NUM  23
    #define PCLK_GPIO_NUM  22

public:
    static const int captureInterval = 100; // Интервал между кадрами в мс (10 FPS)
public:
    Camera()
    {
    }
}
```

Создадим метод *cameraConfig()*, который будет настраивать и готовить ее к работе:


```

void cameraConfig()
{
    camera_config_t config;
    config.ledc_channel = LEDC_CHANNEL_0;
    config.ledc_timer = LEDC_TIMER_0;
    config.pin_d0 = Y2_GPIO_NUM; config.pin_d1 = Y3_GPIO_NUM;
    config.pin_d2 = Y4_GPIO_NUM; config.pin_d3 = Y5_GPIO_NUM;
    config.pin_d4 = Y6_GPIO_NUM; config.pin_d5 = Y7_GPIO_NUM;
    config.pin_d6 = Y8_GPIO_NUM; config.pin_d7 = Y9_GPIO_NUM;
    config.pin_xclk = XCLK_GPIO_NUM; config.pin_pclk = PCLK_GPIO_NUM;
    config.pin_vsync = VSYNC_GPIO_NUM; config.pin_href = HREF_GPIO_NUM;
    config.pin_sscb_sda = SIOD_GPIO_NUM; config.pin_sscb_scl = SIOC_GPIO_NUM;
    config.pin_pwdn = PWDN_GPIO_NUM; config.pin_reset = RESET_GPIO_NUM;
    config.xclk_freq_hz = 20000000;
    config.pixel_format = PIXFORMAT_JPEG;

    // ПЕРЕХОДИМ НА ПОЛНОЕ РАЗРЕШЕНИЕ
    config.frame_size = FRAMESIZE_VGA; // 640x480
    config.jpeg_quality = 12; // 10-12 — хороший баланс четкости
    config.fb_count = 2; // 2 буфера для VGA обязательны

    // Отключаем детектор понижения питания (важно!)
    WRITE_PERI_REG(RTC_CNTL_BROWN_OUT_REG, 0);

    esp_err_t err = esp_camera_init(&config);
    if (err != ESP_OK) {
        Serial.printf("Camera init failed: 0x%x", err);
        return;
    }

    // Корректировки для OV3660
    sensor_t * s = esp_camera_sensor_get();
    if (s) {
        s->set_vflip(s, 1); // Вертикальный переворот
        s->set_hmirror(s, 0); // Горизонтальное отзеркаливание (чтобы право было правом)
    }

    Serial.println("Camera initialized!");
}

```

Вначале функции мы заполняем структуру *camera_config_t*, которая хранит параметры настройки камеры - пины, формат изображения, скорость работы и т.д. Далее передаем эту структуру в функцию *esp_camera_init()*,

которая настраивает камеру и проверяет ее работу. И в конце мы переворачиваем изображение камеры по горизонтальной и вертикальной осям, чтобы изображение было корректным.

Теперь добавим метод, который будет нам выдавать изображение:

```
camera_fb_t* getCupture()
{
    return esp_camera_fb_get();
}
```

В нем мы будем вызывать функцию *esp_camera_fb_get()*, и возвращать изображение, как результат работы функции.

Все методы и классы готовы, приступим к написанию основной логики работы приложения.

3. Логика работы приложения

Переходим в наш основной файл. Первое, что мы сделаем, это определим переменные, которые будем использовать позже:

```
WifiManager wifi; // Объявляем объект класса WifiManager
Camera camera; // Объявляем объект класса Camera

unsigned long lastCaptureTime = 0; // Отчет времени
int frameCount = 0; // Количество полученных кадров
```

В функции *setup()* вызовем функции для настройки камеры и Wi-Fi:

```
camera.cameraConfig();
wifi.connect();
```

Функцию *loop()* реализуем следующим образом:

```

void loop() {
    unsigned long currentTime = millis();

    // Захват кадра по таймеру
    if (currentTime - lastCaptureTime >= Camera::captureInterval) {
        lastCaptureTime = currentTime;

        camera_fb_t* fb = camera.getCupture();
        if (!fb) {
            Serial.println("Ошибка захвата кадра");
            return;
        }

        frameCount++;

        wifi.frameCount = frameCount;
        wifi.process(WifiManager::SendFormat::CAMERA_DATA, fb);

        // ОБЯЗАТЕЛЬНО освобождаем буфер
        esp_camera_fb_return(fb);

        // Статистика
        if (frameCount % 50 == 0) {
            Serial.printf("\nСтатистика: отправлено %d кадров\n", frameCount);
            Serial.printf("Свободная куча: %d байт\n", ESP.getFreeHeap());
            Serial.printf("Свободная PSRAM: %d байт\n\n", ESP.getFreePsram());
        }
    }

    delay(5000); // Только для тестов
}

```

Суть работы такова: ждем 100 мс (`if (currentTime - lastCaptureTime >= Camera::captureInterval)`) после чего получаем изображение (`camera_fb_t* fb = camera.getCupture()`). Далее увеличиваем счетчик кадров и отправляем полученное изображение на сервер с вызовом нужно функции (`wifi.process(WifiManager::SendFormat::CAMERA_DATA, fb)`), после чего освобождаем буфер с кадрами (`esp_camera_fb_return(fb)`). Далее идет необязательный блок вывода статистики в консоль.